Vizja Tech E-commerce: Technical Documentation

Technical Architecture & System Design

1. Project Overview

The Vizja Tech E-commerce Platform is a robust, full-stack web application developed using the Django framework. The project is designed to provide a seamless online shopping experience, featuring an AI-powered semantic search engine, secure user authentication, and a structured order management system. The application follows the Model-Template-View (MTV) architectural pattern.

2. System Architecture

The system is built on a modular architecture where each core functionality is isolated into specific Django apps:

- **Accounts Module (accounts/):** Manages user lifecycle and profile management.
- **Store Module (store/):** The primary engine for product display, Natural Language Processing (NLP) based search, and product rendering.
- **Category Module (category/):** Provides hierarchical organization for products.
- **Cart & Order Modules (carts/, orders/):** Manages product selection and final purchase records.

3. **Technology Stack**

- **Backend Framework:** Python 3.x / Django
- **Database:** PostgreSQL (with pgvector extension)
- **AI Model:** Sentence-Transformers (all-MiniLM-L6-v2) for Semantic Vector Embeddings.
- **Frontend:** Django Template Language (DTL), HTML5, CSS3, JavaScript.

4. **Data Models and Schema**

- **Account:** Stores hashed credentials and user metadata.
- **Product:** Contains pricing, stock levels, and a 384-dimensional Vector Embedding field for AI-driven retrieval.
- **CartItem:** Links products to specific user sessions.
- **Order:** Captures a snapshot of transactions.

Operations, Deployment & Maintenance

1. **Installation and Environment Setup**

- **Virtual Environment:** Initialize a Python virtual environment to isolate dependencies.
- **Dependency Installation:** Install required packages via pip install -r requirements.txt (includes torch, sentence-transformers, and pgvector).

- **Database Initialization:** Ensure PostgreSQL is active with the pgvector extension enabled, then run python manage.py migrate.
- **AI Embedding Generation:** To initialize search functionality, run the save() loop in the Django shell to generate vectors for all products.

2. **Administrative Management The Django Admin Interface is used for:**
   - **Inventory Updates:** Adding or editing products (which automatically triggers AI re-indexing).
   - **Order Tracking:** Reviewing customer orders.

3**. Security & Scalability Protocols**
   - **Data Security:** All passwords are encrypted using PBKDF2. CSRF protection is enabled.
   - **Vector Search Logic:** The search uses Cosine Similarity to match user intent to product features, providing higher accuracy than standard text matching.
   - **Fuzzy Logic:** Integrated Levenshtein Distance spell-checking to handle user typos in the hardware catalog.

4. **Troubleshooting Guide**
   - **Issue:** Search results not appearing or irrelevant.
   - **Solution:** Ensure the p.save() loop has been run in the shell to populate the embedding column. Verify the distance__lt threshold in views.py.
   - **Issue:** ImportError regarding torch or sentence_transformers.
   - **Solution:** Re-run pip install -r requirements.txt to ensure all AI dependencies are present.
   - **Issue:** Database connection failed.
   - **Solution:** Check DATABASES settings in settings.py to ensure it is pointing to the PostgreSQL service rather than SQLite

.